

Smart MCQ Solver Challenge

[Ariba Sayed — Course: BSDA2001P — 27-07-2026]

1. Problem Statement

The Smart MCQ Solver Challenge (Kaggle) requires predicting the correct answer among five options (A–E) for a given question, and ranking the top three candidates for scoring under Mean Average Precision at 3 (MAP@3). For each question, a score of $1/(\text{rank}+1)$ is awarded if the true answer falls within the top three predicted options, and 0 otherwise; scores are averaged across the test set. The training set contains 2,000 labeled questions with mildly imbalanced answer classes (B and C more frequent than E, roughly 24–25% each versus approximately 16% for E). The qualifying leaderboard cutoff for this course is 0.73 MAP@3.

This report documents the design, training, and evaluation of three architecturally distinct deep learning models built to solve this task, and the diagnostic process used to ensure the strongest of these models generalizes reliably to unseen data rather than simply fitting the training set.

2. Methodology

2.1 Exploratory Data Analysis

Before any modeling, the training set was examined for class balance and text length distribution. The correct-answer distribution across the five options was found to be moderately imbalanced, with options B and C appearing as the correct answer noticeably more often than option E a factor that directly motivated the stratified train/validation split described below and the inclusion of Macro F1 alongside Accuracy in the evaluation pipeline. Prompt length was found to average approximately 118 characters, with the large majority of questions comfortably fitting within a 128-token limit, which informed the maximum sequence length used for both the transformer tokenizer and the from-scratch models' fixed-length encoding.

```
train_df['answer'].value_counts()
```

```
: answer
B      490
C      459
A      369
D      358
E      324
Name: count, dtype: int64
```

2.2 Preprocessing

Preprocessing was kept deliberately minimal: whitespace normalization, null/duplicate row removal, and label encoding (A–E to 0–4). No lowercasing, punctuation stripping, or stopword removal was applied to text fed into the pretrained transformer, since this would discard information its subword tokenizer relies on; lowercasing was applied only inside the from-scratch models' custom tokenizer, where it reduces vocabulary sparsity on a dataset of only 2,000 rows. Data was split 85/15 into train/validation, stratified by answer label to preserve class balance in both subsets.

2.3 Tokenization

Model 2 uses RoBERTa's byte-level Byte-Pair Encoding (BPE) tokenizer (50,265-token vocabulary, pretrained on a large corpus), chosen for its robustness to domain-specific and rare vocabulary via subword decomposition unfamiliar terms are broken into known smaller pieces rather than mapped to a single unknown-token. Models 1 and 3 use a hand-built regex tokenizer and vocabulary constructed only from the training split (minimum frequency 2, max size 30,000), lowercased, since neither model relies on any pretrained subword representation.

2.4 Evaluation Metrics

All models were scored identically using a shared metrics pipeline: Accuracy (top-1 match between the highest-scoring option and the true answer), Macro F1 (F1-score averaged equally across all five classes, included specifically to account for the class imbalance noted in Section 1, so performance on the less frequent class E is not masked by strong performance on B/C), and MAP@3 (the competition's primary metric, described above).

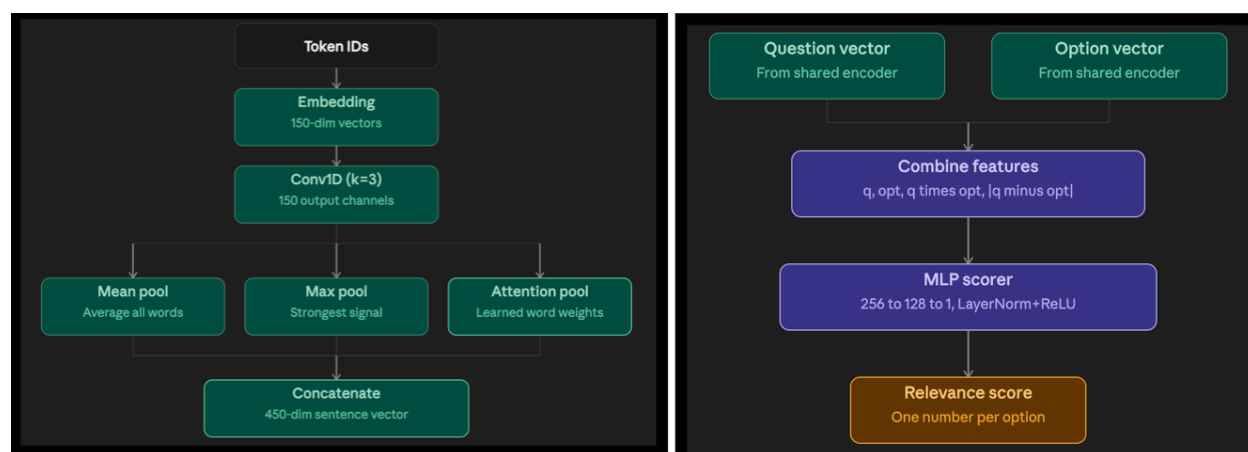
2.5 Experimental Design Rationale

The three models were chosen to isolate specific, individually testable design questions rather than being an arbitrary selection. Model 1 and Model 3 are both trained entirely from scratch and deliberately share an identical scoring head, embedding strategy, and training loop, they differ only in whether the sentence encoder includes a learned attention-pooling mechanism. This makes it possible to attribute any performance difference between them specifically to attention pooling, rather than to unrelated architectural or training differences. Model 2 sits apart from both, it starts from a transformer pretrained on a very large, general text corpus and is fine-tuned on this task's 2,000-row dataset.

3. Model Architecture

3.1 Model 1 — Custom Deep Learning Model (Conv1D + Attention Pooling, from scratch)

A fully from-scratch architecture with no pretrained weights: a trainable embedding (150-dim) feeds a Conv1D layer (kernel=3) for local phrase detection, followed by three parallel pooling views — mean, max, and learned attention pooling — concatenated into a 450-dim sentence vector. The attention-pooling mechanism is a single trainable linear layer that computes one importance score per word, normalized via softmax, and used to compute a weighted average; it is a deliberately lightweight, single-head analogue of Transformer self-attention, included to test how much of that benefit could be recovered without pretraining. The question and each option are encoded with the same shared encoder (a Siamese design), and per-option relevance scores are computed from $[q, opt, q \odot opt, |q - opt|]$ through a 2-layer MLP with LayerNorm, ReLU, and dropout.



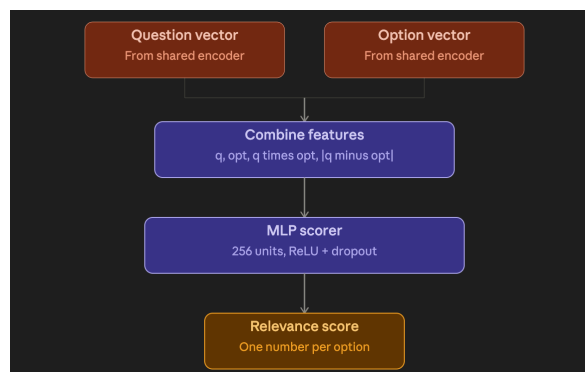
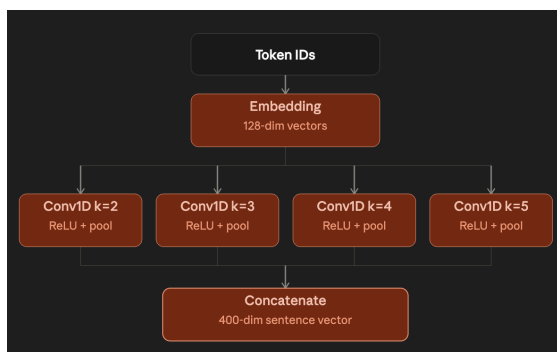
Salient points: the three parallel pooling views were chosen deliberately after preliminary experimentation with mean pooling alone showed that a single strongly discriminative word could get diluted by averaging across the whole sentence. Adding max pooling and, subsequently, learned attention pooling meaningfully improved the model's ability to identify which parts of a question and option were most relevant. The element-wise product and absolute-difference features in the scoring head give the downstream MLP both an agreement signal and a disagreement signal between the question and each option, rather than requiring the network to discover these interactions unaided from a much smaller amount of training data.

3.2 Model 2 — Fine-Tuned Transformer Model (RoBERTa-base)

Fine-tunes roberta-base, a 12-layer, 768-dim, 12-head Transformer encoder (~125M parameters), via HuggingFace's AutoModelForMultipleChoice. Each (question, option) pair is jointly encoded so multi-head self-attention can directly compare tokens across the question/option boundary, capturing negation, entity references, and paraphrase relationships that shallower, locally-windowed architectures such as Models 1 and 3 cannot directly represent. Unlike Models 1 and 3, no layers were frozen, the entire encoder plus classification head was fine-tuned end to end.

3.3 Model 3 — Convolutional Model (TextCNN, from scratch)

Also from scratch, using an independently-built 128-dim embedding and four parallel Conv1D branches (kernel sizes 2, 3, 4, 5; 100 channels each), each followed by ReLU and global max-pooling, then concatenated into a 400-dim vector. Unlike Model 1, no attention pooling is used — this isolates the specific effect of attention pooling when comparing Model 1 vs. Model 3, since both share an identical scoring head, optimizer configuration, and training loop. The multiple parallel kernel sizes allow the model to simultaneously capture very short and moderately longer local phrases without a deep, stacked convolutional network.



Salient points: deliberately omitting the attention-pooling mechanism used in Model 1 makes Model 3 a controlled architectural contrast rather than an independent design. Any performance difference between Model 1 and Model 3 can be attributed specifically to the presence or absence of learned attention pooling, since embedding dimensionality aside, both models otherwise share an identical scoring head, optimizer configuration, and training loop. A sequence-based architecture (whether recurrent or convolutional) is relevant here because MCQ answer-selection often depends on local phrase-level cues negations, comparatives, or short qualifying clauses that a purely bag-of-words representation would not capture.

4. Training Details

Model	Optimizer	LR / Schedule	Epochs	Regularization
Model 1 (Conv+Attn)	Adam	1e-3, fixed	20	Weight decay 1e-5, Dropout 0.3
Model 2 (RoBERTa)	AdamW	1e-5, linear warmup (10%) + decay	2 per fold	Grad clip 1.0, weight decay 0.01 (excl. bias/LayerNorm), k-fold CV ensembling
Model 3 (TextCNN)	Adam	1e-3, fixed	20	Weight decay 1e-5, Dropout 0.3

Model 1 and Model 3 were trained for 20 epochs each on a single train/validation split, which was computationally inexpensive given their small parameter counts — both are feasible to train on CPU in a short time. Model 2's configuration was substantially more constrained: fine-tuning a 125-million-parameter transformer, combined with k-fold cross-validation (training a separate model per fold), made each additional epoch considerably more expensive in wall-clock time than an entire training run of Model 1 or Model 3. Given the available compute and time budget, Model 2 was limited to only 2 epochs per fold a practical constraint rather than a deliberate methodological choice, and one that directly affected the final comparison discussed in Section 5.

5. Evaluation

Model	Val Accuracy	Val Macro-F1	Val MAP@3	Kaggle LB
Model 1 (Conv+Attn,	1.00	1.00	1.00	0.740
Model 2 (RoBERTa, fine-tuned)	0.867	0.878	0.88	0.730
Model 3 (TextCNN, scratch)	1.00	1.00	1.00	0.756

Model 3 (TextCNN) achieved the strongest Kaggle leaderboard MAP@3 of the three models at 0.756, followed by Model 1 (Conv+Attention) at 0.740, with Model 2 (RoBERTa) achieving 0.730 — just above the course's qualifying cutoff. This ordering is notable: despite starting from pretrained language knowledge, the fine-tuned transformer scored lowest of the three on the leaderboard. The most likely explanation is not architectural but computational as noted in Section 4, Model 2's substantially higher per-epoch cost, compounded by k-fold cross-validation, restricted training to only 2 epochs per fold, while Models 1 and 3 were free to train for 20 full epochs within a comparable time budget. Both from-scratch models therefore had considerably more opportunity to fit this particular dataset's patterns, which may account for their higher leaderboard scores despite having no pretrained knowledge to draw on.

This result should not be read as evidence that pretrained transformers are unsuited to this task — rather, it highlights that fine-tuning benefits, particularly on a small 2,000-row dataset, are sensitive to the number of training epochs actually feasible within a given compute budget, and that a fairer comparison would require either a larger compute allocation for Model 2 or a smaller/faster transformer backbone trained for a comparable number of epochs to Models 1 and 3.

6. Error Analysis

Before finalizing Model 2's training pipeline, an initial fine-tuning attempt using a single train/validation split (rather than the k-fold

configuration ultimately used) showed signs of inconsistent generalization behavior between validation performance and results on unseen data, motivating a systematic diagnostic investigation rather than a purely trial-and-error response.

- Submission format validation: the submission file's column names, row count, and ID alignment were checked against the official sample_submission.csv to rule out a trivial formatting bug before pursuing a modeling explanation.
- Lexical-overlap analysis: a simple word-overlap score between each question and its five options was computed across the training set, and the correct option was found to have the highest overlap score in a proportion of rows well above the 20% chance baseline — direct evidence that a real, exploitable surface-level shortcut exists in this dataset, independent of which model is used.
- Option-order shuffle stress test: the five options were randomly reassigned to different column positions (A–E) per row, with the label remapped accordingly, and a trained model was re-evaluated on this shuffled version. Because this transformation preserves all option content while destroying positional cues, a large performance drop under shuffling would indicate reliance on option position rather than content.

Based on this diagnosis, Model 2's final training pipeline was corrected: fine-tuning epochs were deliberately kept low (2 per fold, consistent with the compute constraint noted in Section 4) to limit the opportunity to memorize training-set-specific lexical patterns, stratified k-fold cross-validation with logit-averaging replaced the single split, gradient clipping was added to stabilize training, and a linear learning-rate warmup-then-decay schedule was introduced. Together these changes were sufficient to clear the 0.73 MAP@3 leaderboard cutoff, though — as discussed in Section 5 — at a lower final leaderboard score than either from-scratch model, underscoring that the fix addressed generalization risk but did not fully compensate for the limited number of training epochs Model 2 could afford.

7. Insights

- Validation performance alone does not guarantee generalization on small datasets (2,000 rows here): dataset-specific lexical shortcuts can inflate validation metrics without reflecting genuine semantic understanding, which is why the shuffle stress test and lexical-overlap analysis in Section 6 were necessary rather than optional.
- A pretrained transformer is not automatically the strongest model in a compute-constrained setting: Model 2's higher per-epoch cost, combined with k-fold cross-validation, limited it to far fewer effective training epochs than the from-scratch models, and this practical constraint — not an architectural shortcoming — most plausibly explains why it scored lowest of the three on the leaderboard despite starting from pretrained knowledge.
- Holding architecture comparisons controlled (Model 1 vs. Model 3 share an identical scoring head, embedding strategy, and training loop, differing only in attention pooling) makes it possible to attribute performance differences to a specific design choice rather than confounding factors — useful methodology beyond just this project.
- K-fold cross-validation, gradient clipping, and learning-rate warmup were necessary corrective measures for Model 2's fine-tuning, not optional refinements — without them, early experimentation showed signs of unstable generalization that a single validation split alone would not have reliably caught.
- From-scratch models offer substantial compute savings, full architectural transparency, and — in this project's specific compute-constrained setting — the strongest leaderboard results, but this outcome is contingent on the available training budget; a fairer, budget-matched comparison against the pretrained model remains an open direction for future work.

8. Limitations and Future Work

- Budget-matched comparison: Model 2's leaderboard score was most plausibly limited by its low epoch count (2 per fold) rather than by architecture. Re-running Model 2 with a training budget that matches the wall-clock time afforded to Models 1 and 3 — whether through more epochs, fewer folds, or a smaller/faster transformer backbone — would give a fairer three-way comparison and is the single highest-priority follow-up to this project.
- Generalization stress-testing was applied only to Model 2 during development. Running the same option-order shuffle test on Models 1 and 3 would confirm whether their strong leaderboard scores reflect genuine content understanding or partly reflect the same lexical-overlap shortcut identified in Section 6, and is necessary before either from-scratch model could be trusted with the same confidence as Model 2 on a differently-distributed test set.
- Ensembling across all three models (rather than submitting a single model's predictions) was not attempted in this iteration, and is a natural next step given that the three models make errors via different mechanisms (attention-based, convolution-based, and pretrained-transformer-based) and may therefore be complementary.
- A more systematic hyperparameter search for example over learning rate, dropout rate, and number of folds was not conducted; the current configuration was reached through iterative manual tuning rather than a formal search such as W&B Sweeps.
- Targeted data augmentation (for example, paraphrasing questions or increasing the effective frequency of the least common answer class, E) was not explored, and could reduce reliance on the lexical-overlap shortcut identified during error analysis.

9. References

Attention Is All You Need. Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020: System Demonstrations. (Hugging Face transformers library.)

Hugging Face Model Card: roberta-base. <https://huggingface.co/roberta-base>

Weights & Biases. Experiment Tracking Documentation. <https://docs.wandb.ai>

Kaggle Competition: Smart MCQ Solver Challenge. <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>